Fórum de Discussão

Trabalho de Padrões de Projeto (Design Patterns) — implementação em Python

---

1. Visão geral

O projeto implementa, em Python 3 e sem nenhuma dependência externa, um fórum de discussão operado por um menu de terminal. O objetivo não é o fórum em si, e sim usar um único domínio coerente para aplicar **nove padrões de projeto do catálogo GoF** — três criacionais, três estruturais e três comportamentais — cada um resolvendo um problema real da aplicação.

Para que o papel de cada padrão fique visível durante a execução, o programa imprime uma marca entre colchetes sempre que um padrão entra em ação: `[Singleton]`, `[Builder]`, `[Factory Method]`, `[Composite]`, `[Decorator]`, `[Facade]`, `[Observer]`, `[Strategy]` e `[State]`. Essas marcas aparecem na bateria de testes da seção 4.

1.1 Padrões implementados

#	Padrão	Categoria	Onde está	Papel no fórum
1	Singleton	Criacional	`forum/criacional/singleton.py`	Repositório único do fórum: usuários, tópicos e contador de identificadores
2	Factory Method	Criacional	`forum/criacional/factory_method.py`	Roteiro único de publicação que fabrica Tópico, Mensagem ou Anexo
3	Builder	Criacional	`forum/criacional/builder.py`	Montagem passo a passo de um tópico (título, autor, texto, tags, fixado)
4	Composite	Estrutural	`forum/estrutural/composite.py`	Árvore da discussão: tópico → mensagens → respostas → anexos
5	Decorator	Estrutural	`forum/estrutural/decorator.py`	Filtros de conteúdo empilháveis: censura, destaque e assinatura
6	Facade	Estrutural	`forum/estrutural/facade.py`	Interface única de alto nível usada pelo menu
7	Observer	Comportamental	`forum/comportamental/observer.py`	Notificação dos usuários inscritos em um tópico
8	Strategy	Comportamental	`forum/comportamental/strategy.py`	Algoritmos de ordenação da lista de tópicos
9	State	Comportamental	`forum/comportamental/state.py`	Estados do tópico: Aberto, Fechado e Arquivado

1.2 Estrutura de diretórios

```
forum-design-patterns/
├── main.py                       menu interativo (cliente do subsistema)
├── README.md
├── forum/
│   ├── __init__.py
│   ├── modelos.py                Usuario
│   ├── criacional/
│   │   ├── singleton.py          Forum
│   │   ├── factory_method.py     CriadorDePublicacao e criadores concretos
│   │   └── builder.py            ConstrutorDeTopico
│   ├── estrutural/
│   │   ├── composite.py          ItemDiscussao, Anexo, Mensagem, Topico
│   │   ├── decorator.py          Conteudo e decoradores de texto
│   │   └── facade.py             ForumFacade
│   └── comportamental/
│       ├── observer.py           Assunto, Observador e observadores concretos
│       ├── strategy.py           EstrategiaDeOrdenacao e ListagemDeTopicos
│       └── state.py              EstadoTopico, Aberto, Fechado, Arquivado
├── testes/
│   ├── test_padroes.py           29 testes automatizados (unittest)
│   └── roteiro_bateria.txt       entradas da bateria de testes da seção 4
└── docs/
    └── DOCUMENTACAO.md           este documento
```

1.3 Como executar

```
python main.py                                # menu interativo
python main.py < testes/roteiro_bateria.txt   # reproduz a bateria da seção 4
python -m unittest discover -s testes -v      # testes automatizados
```

Requisitos: Python 3.9 ou superior. Nenhuma biblioteca externa é utilizada.

---

2. Os nove padrões aplicados ao fórum

2.1 Singleton (criacional)

Intenção. Garantir que uma classe tenha uma única instância e fornecer um ponto global de acesso a ela.

Problema no fórum. O cadastro de usuários, a coleção de tópicos e o contador de identificadores precisam ser únicos. Se cada parte do sistema criasse o seu próprio objeto `Forum`, dois tópicos diferentes poderiam receber o identificador `#1` e um usuário cadastrado em um ponto não existiria no outro.

Implementação. O controle fica em `__new__`: a primeira chamada cria o objeto e o guarda em `Forum._instancia`; as chamadas seguintes devolvem o mesmo objeto. O método de classe `resetar()` existe apenas para isolar os testes automatizados.

2.2 Factory Method (criacional)

Intenção. Definir uma interface para criar um objeto, deixando que as subclasses decidam qual classe instanciar.

Problema no fórum. Publicar um item segue sempre o mesmo roteiro — anunciar, fabricar, confirmar — mas o produto varia: `Topico`, `Mensagem` ou `Anexo`.

Implementação. `CriadorDePublicacao.publicar()` concentra o roteiro comum e chama `_fabricar()`, o Factory Method, sobrescrito por `CriadorDeTopico`, `CriadorDeMensagem` e `CriadorDeAnexo`. O dicionário `CRIADORES` permite obter o criador pelo nome do tipo, sem `if` encadeado.

2.3 Builder (criacional)

Intenção. Separar a construção de um objeto complexo da sua representação.

Problema no fórum. Abrir um tópico envolve cinco informações, sendo três opcionais (texto de abertura, tags e sinalização de fixado). Um construtor com cinco parâmetros posicionais seria ilegível e frágil.

Implementação. `ConstrutorDeTopico` expõe métodos encadeáveis (`com_titulo`, `com_autor`, `com_tags`, `fixado`), valida o que é obrigatório em `construir()` e só então delega a fabricação ao Factory Method. Após entregar o produto, o construtor se reinicia, evitando que sobras de um tópico vazem para o próximo.

2.4 Composite (estrutural)

Intenção. Compor objetos em estruturas de árvore e tratar objetos individuais e composições de maneira uniforme.

Problema no fórum. Uma discussão é uma árvore: o tópico contém mensagens, cada mensagem contém respostas e anexos, sem limite de profundidade. Exibir, contar e percorrer precisam funcionar igual em qualquer nível.

Implementação. `ItemDiscussao` é o componente e define `exibir()`, `total_itens()`, `caminho()` e `percorrer()` de forma recursiva. `Anexo` é a folha e recusa filhos com `TypeError`; `Mensagem` e `Topico` são compostos. O método `caminho()` reconstrói o endereço do nó até a raiz (por exemplo `/T1/M3/M4/A5`), do mesmo modo que a função `mostrarCaminho` do trabalho em C.

2.5 Decorator (estrutural)

Intenção. Anexar responsabilidades a um objeto dinamicamente, como alternativa flexível à herança.

Problema no fórum. O texto publicado pode passar por censura de palavras proibidas, destaque de moderação e assinatura do autor, em qualquer combinação e ordem. Criar uma subclasse para cada combinação seria inviável.

Implementação. `Conteudo` é o componente, `ConteudoSimples` guarda o texto cru e `DecoradorDeConteudo` envolve outro `Conteudo`. Os decoradores concretos são empilhados em tempo de execução; a ordem da pilha altera o resultado final, como demonstram o Teste 12 e o teste automatizado `test_ordem_importa`.

2.6 Facade (estrutural)

Intenção. Fornecer uma interface unificada para um conjunto de interfaces de um subsistema.

Problema no fórum. Abrir uma discussão exige coordenar seis participantes: repositório único, construtor, criador, árvore, observadores e estado inicial. O menu não deve conhecer nada disso.

Implementação. `ForumFacade` oferece operações de alto nível (`abrir_discussao`, `responder`, `buscar`, `alterar_estado`) e esconde a coordenação. O arquivo `main.py` importa apenas a fachada: nenhuma classe de padrão aparece no cliente.

2.7 Observer (comportamental)

Intenção. Definir uma dependência um-para-muitos, de modo que a mudança de estado de um objeto notifique automaticamente seus dependentes.

Problema no fórum. Quem acompanha um tópico precisa ser avisado a cada nova resposta, e o tópico não pode depender da forma de entrega do aviso.

Implementação. `Topico` herda de `Assunto` e mantém a lista de observadores. `NotificadorDeUsuario` entrega o aviso na caixa de entrada de um usuário; `RegistroDeAtividade` grava o evento para auditoria. Os dois implementam a mesma interface `Observador`, e o tópico não sabe a diferença entre eles. A comparação por `__eq__` impede inscrições duplicadas do mesmo usuário.

2.8 Strategy (comportamental)

Intenção. Definir uma família de algoritmos, encapsulá-los e torná-los intercambiáveis.

Problema no fórum. A lista de tópicos pode ser ordenada por mais recentes, mais respondidos ou mais votados, e a escolha muda em tempo de execução.

Implementação. Cada algoritmo é uma classe que implementa `ordenar()`. O contexto `ListagemDeTopicos` apenas delega à estratégia configurada e pode trocá-la a qualquer momento com `definir_estrategia()`, sem nenhum `if` sobre o tipo de ordenação.

2.9 State (comportamental)

Intenção. Permitir que um objeto altere seu comportamento quando seu estado interno muda.

Problema no fórum. Um tópico aberto aceita respostas; fechado, não; arquivado, nunca mais volta atrás. Codificar isso com condicionais espalharia a mesma regra por vários métodos.

Implementação. `EstadoTopico` declara `pode_receber_resposta()` e as transições `abrir`, `fechar` e `arquivar`. Cada estado concreto autoriza apenas as transições válidas e recusa as demais; `Arquivado` é terminal. O tópico delega todas as decisões ao objeto de estado atual.

Transições válidas.

```
Aberto    --fechar-->    Fechado
Aberto    --arquivar-->  Arquivado
Fechado   --abrir-->     Aberto
Fechado   --arquivar-->  Arquivado
Arquivado --(nenhuma)->  estado terminal
```

---

3. Código-fonte completo (Python)

Arquivo: `main.py`

```python
"""Forum de Discussao - programa principal (menu interativo).

Trabalho de Design Patterns: nove padroes GoF aplicados a um unico
dominio. O menu abaixo e o cliente do subsistema e conversa apenas com a
fachada (ForumFacade), sem conhecer os padroes internos.

    Criacionais     Singleton, Factory Method, Builder
    Estruturais     Composite, Decorator, Facade
    Comportamentais Observer, Strategy, State

Execucao:
    python main.py
"""

from forum.estrutural.facade import ForumFacade

MENU = """
===== Forum de Discussao =====
1.  Cadastrar usuario                    [Singleton]
2.  Abrir novo topico                    [Builder + Factory Method]
3.  Responder a um topico ou mensagem    [Factory Method + State + Observer]
4.  Anexar arquivo a uma mensagem        [Composite - folha]
5.  Exibir a arvore de uma discussao     [Composite]
6.  Listar topicos                       [Strategy]
7.  Inscrever usuario em um topico       [Observer]
```

```
        8.  Alterar o estado de um topico          [State]
        9.  Votar em uma mensagem
        10. Formatar um conteudo                   [Decorator]
        11. Buscar por texto                       [Facade]
        12. Ver caixa de entrada de um usuario     [Observer]
        13. Estatisticas do forum                  [Singleton]
        0.  Sair
        Escolha uma opcao: """


    def ler_texto(rotulo):
        return input(rotulo).strip()


    def ler_inteiro(rotulo):
        """Le um numero inteiro; devolve None quando a entrada e invalida."""
        bruto = input(rotulo).strip()
        try:
            return int(bruto)
        except ValueError:
            print(f'Erro: "{bruto}" nao e um numero inteiro valido.')
            return None


    def cadastrar_usuario(fachada):
        print("\n--- Cadastrar Usuario ---")
        apelido = ler_texto("Apelido: ")
        email = ler_texto("E-mail: ")
        papel = ler_texto("Papel (membro/moderador): ") or "membro"
        fachada.cadastrar_usuario(apelido, email, papel)


    def abrir_topico(fachada):
        print("\n--- Abrir Novo Topico ---")
        autor = ler_texto("Apelido do autor: ")
        titulo = ler_texto("Titulo: ")
        conteudo = ler_texto("Texto de abertura: ")
        tags = [t.strip() for t in ler_texto("Tags (separadas por virgula): ").split(",") if t.strip()]
        fixado = ler_texto("Fixar o topico? (s/n): ").lower().startswith("s")
        fachada.abrir_discussao(autor, titulo, conteudo, tags, fixado)


    def responder(fachada):
        print("\n--- Responder ---")
        identificador = ler_inteiro("Identificador do topico ou da mensagem (#): ")
        if identificador is None:
            return
        autor = ler_texto("Apelido do autor: ")
        texto = ler_texto("Resposta: ")
        fachada.responder(identificador, autor, texto)


    def anexar(fachada):
        print("\n--- Anexar Arquivo ---")
        identificador = ler_inteiro("Identificador da mensagem (#): ")
        if identificador is None:
            return
        autor = ler_texto("Apelido do autor: ")
        nome = ler_texto("Nome do arquivo: ")
        tamanho = ler_inteiro("Tamanho em KB: ")
        if tamanho is None:
            return
        fachada.anexar(identificador, autor, nome, tamanho)


    def exibir_discussao(fachada):
        print("\n--- Exibir Arvore da Discussao ---")
        identificador = ler_inteiro("Identificador do topico (#): ")
        if identificador is None:
            return
        fachada.exibir_discussao(identificador)


    def listar_topicos(fachada):
        print("\n--- Listar Topicos ---")
        chave = ler_texto("Ordenar por (recentes/respondidos/votados): ") or "recentes"
        fachada.listar_topicos(chave)


    def inscrever(fachada):
        print("\n--- Inscrever Usuario em Topico ---")
        identificador = ler_inteiro("Identificador do topico (#): ")
        if identificador is None:
            return
        apelido = ler_texto("Apelido do usuario: ")
        fachada.inscrever(identificador, apelido)


    def alterar_estado(fachada):
        print("\n--- Alterar Estado do Topico ---")
        identificador = ler_inteiro("Identificador do topico (#): ")
        if identificador is None:
            return
        acao = ler_texto("Acao (abrir/fechar/arquivar): ")
        fachada.alterar_estado(identificador, acao)


    def votar(fachada):
        print("\n--- Votar em Mensagem ---")
        identificador = ler_inteiro("Identificador da mensagem (#): ")
```

```
            if identificador is None:
                return
            fachada.votar(identificador)


        def formatar(fachada):
            print("\n--- Formatar Conteudo ---")
            texto = ler_texto("Texto original: ")
            filtros = [
                f.strip().lower()
                for f in ler_texto("Filtros (censura/destaque/assinatura, por virgula): ").split(",")
                if f.strip()
            ]
            apelido = ler_texto("Apelido para assinatura (enter se nao usar): ") or None
            fachada.formatar(texto, filtros, apelido)


        def buscar(fachada):
            print("\n--- Buscar por Texto ---")
            termo = ler_texto("Termo: ")
            fachada.buscar(termo)


        def caixa_de_entrada(fachada):
            print("\n--- Caixa de Entrada ---")
            apelido = ler_texto("Apelido do usuario: ")
            fachada.caixa_de_entrada(apelido)


        def estatisticas(fachada):
            print("\n--- Estatisticas ---")
            fachada.estatisticas()


        OPCOES = {
            1: cadastrar_usuario,
            2: abrir_topico,
            3: responder,
            4: anexar,
            5: exibir_discussao,
            6: listar_topicos,
            7: inscrever,
            8: alterar_estado,
            9: votar,
            10: formatar,
            11: buscar,
            12: caixa_de_entrada,
            13: estatisticas,
        }


        def main():
            print("Inicializando o forum...")
            fachada = ForumFacade()
            print("Forum pronto. Nenhum usuario e nenhum topico cadastrados.")

            while True:
                try:
                    escolha = input(MENU).strip()
                except EOFError:
                    print("\nEntrada encerrada. Saindo.")
                    break

                if escolha == "0":
                    print("\nEncerrando o programa.")
                    print("A instancia unica do Forum sera descartada pelo coletor de lixo.")
                    break

                try:
                    opcao = int(escolha)
                except ValueError:
                    print(f'\nOpcao invalida: "{escolha}".')
                    continue

                acao = OPCOES.get(opcao)
                if acao is None:
                    print(f"\nOpcao invalida: {opcao}.")
                    continue

                try:
                    acao(fachada)
                except EOFError:
                    print("\nEntrada encerrada. Saindo.")
                    break


        if __name__ == "__main__":
            main()
```

Arquivo: `forum/modelos.py`

```
        """Modelos de dominio que nao pertencem a nenhum padrao especifico."""


        class Usuario:
            """Participante do forum.
```

```
    A caixa de entrada e preenchida pelos observadores (padrao Observer)
    sempre que um topico assinado pelo usuario recebe uma nova resposta.
    """

    PAPEIS = ("membro", "moderador")

    def __init__(self, apelido, email, papel="membro"):
        if papel not in self.PAPEIS:
            raise ValueError(f'Papel invalido: "{papel}". Use um de {self.PAPEIS}.')
        self.apelido = apelido
        self.email = email
        self.papel = papel
        self.caixa_entrada = []

    @property
    def e_moderador(self):
        return self.papel == "moderador"

    def __str__(self):
        return f"{self.apelido} ({self.papel})"

    def __repr__(self):
        return f"Usuario({self.apelido!r}, {self.email!r}, {self.papel!r})"
```

Arquivo: `forum/criacional/singleton.py`

```
"""Padrao SINGLETON (criacional).

Intencao
    Garantir que uma classe tenha uma unica instancia e fornecer um ponto
    global de acesso a ela.

Aplicacao no forum
    Existe um unico forum em execucao. Ele concentra o cadastro de
    usuarios, a colecao de topicos e o contador de identificadores. Se
    cada parte do sistema criasse o seu proprio "Forum", os contadores e
    as colecoes ficariam dessincronizados.

Implementacao
    O controle e feito em __new__: a primeira chamada cria e guarda a
    instancia em Forum._instancia; as seguintes devolvem o mesmo objeto.
"""


class Forum:
    """Repositorio central do forum. Instancia unica."""

    _instancia = None

    def __new__(cls):
        if cls._instancia is None:
            print("[Singleton] Primeira chamada: criando a unica instancia de Forum...")
            cls._instancia = super().__new__(cls)
            cls._instancia._inicializar()
        return cls._instancia

    def _inicializar(self):
        self.nome = "Forum de Discussao"
        self._usuarios = {}
        self._topicos = {}
        self._proximo_identificador = 1

    # --- contador de identificadores ------------------------------------
    def proximo_identificador(self):
        atual = self._proximo_identificador
        self._proximo_identificador += 1
        return atual

    # --- usuarios -------------------------------------------------------
    def registrar_usuario(self, usuario):
        if usuario.apelido in self._usuarios:
            return False
        self._usuarios[usuario.apelido] = usuario
        return True

    def obter_usuario(self, apelido):
        return self._usuarios.get(apelido)

    @property
    def usuarios(self):
        return tuple(self._usuarios.values())

    # --- topicos --------------------------------------------------------
    def adicionar_topico(self, topico):
        self._topicos[topico.identificador] = topico
        return topico

    def obter_topico(self, identificador):
        return self._topicos.get(identificador)

    @property
    def topicos(self):
        return tuple(self._topicos.values())

    # --- consultas sobre a arvore (Composite) ---------------------------
    def localizar(self, identificador):
        """Procura um item (topico, mensagem ou anexo) em todas as arvores."""
```

```
            for topico in self._topicos.values():
                for item in topico.percorrer():
                    if item.identificador == identificador:
                        return item
            return None

        def buscar_por_texto(self, termo):
            """Devolve todos os itens cujo conteudo ou titulo contem o termo."""
            termo = termo.lower()
            encontrados = []
            for topico in self._topicos.values():
                for item in topico.percorrer():
                    alvo = f"{getattr(item, 'titulo', '')} {item.conteudo}".lower()
                    if termo in alvo:
                        encontrados.append(item)
            return encontrados

        # --- estatisticas ------------------------------------------------------
        def estatisticas(self):
            topicos = self.topicos
            return {
                "usuarios": len(self._usuarios),
                "topicos": len(topicos),
                "mensagens": sum(t.total_mensagens() for t in topicos),
                "itens_na_arvore": sum(t.total_itens() for t in topicos),
                "votos": sum(t.total_votos() for t in topicos),
                "proximo_identificador": self._proximo_identificador,
            }

        @classmethod
        def resetar(cls):
            """Descarta a instancia unica. Usado apenas pelos testes."""
            cls._instancia = None

        def __repr__(self):
            return f"<Forum id=0x{id(self):x} topicos={len(self._topicos)}>"
```

Arquivo: `forum/criacional/factory_method.py`

```
"""Padrao FACTORY METHOD (criacional).

Intencao
    Definir uma interface para criar um objeto, mas deixar que as
    subclasses decidam qual classe instanciar. O Factory Method permite
    que uma classe delegue a instanciacao as subclasses.

Aplicacao no forum
    Publicar um item sempre segue o mesmo roteiro (anunciar, fabricar,
    confirmar). O que muda e o produto: um Topico, uma Mensagem ou um
    Anexo. O metodo publicar() e o codigo comum; _fabricar() e o Factory
    Method sobrescrito por cada criador concreto.

Participantes
    Criador (Creator)            -> CriadorDePublicacao
    Criadores concretos          -> CriadorDeTopico, CriadorDeMensagem, CriadorDeAnexo
    Produto (Product)            -> ItemDiscussao
    Produtos concretos           -> Topico, Mensagem, Anexo
"""

from abc import ABC, abstractmethod

from ..estrutural.composite import Anexo, Mensagem, Topico


class CriadorDePublicacao(ABC):
    """Criador: implementa o roteiro de publicacao e declara o Factory Method."""

    tipo = "publicacao"

    @abstractmethod
    def _fabricar(self, identificador, autor, **dados):
        """Factory Method: cada subclasse devolve o seu produto concreto."""

    def publicar(self, identificador, autor, **dados):
        print(f"[Factory Method] {type(self).__name__} vai fabricar um(a) {self.tipo}...")
        produto = self._fabricar(identificador, autor, **dados)
        print(f"[Factory Method] Produto criado: {produto.rotulo()}")
        return produto


class CriadorDeTopico(CriadorDePublicacao):
    tipo = "Topico"

    def _fabricar(self, identificador, autor, **dados):
        return Topico(
            identificador,
            autor,
            dados["titulo"],
            dados.get("conteudo", ""),
            dados.get("tags"),
        )


class CriadorDeMensagem(CriadorDePublicacao):
    tipo = "Mensagem"

    def _fabricar(self, identificador, autor, **dados):
```

```
            return Mensagem(identificador, autor, dados["conteudo"])


        class CriadorDeAnexo(CriadorDePublicacao):
            tipo = "Anexo"

            def _fabricar(self, identificador, autor, **dados):
                return Anexo(
                    identificador,
                    autor,
                    dados["nome_arquivo"],
                    dados.get("tamanho_kb", 0),
                )


        CRIADORES = {
            "topico": CriadorDeTopico,
            "mensagem": CriadorDeMensagem,
            "anexo": CriadorDeAnexo,
        }


        def obter_criador(tipo):
            """Devolve uma instancia do criador registrado para o tipo informado."""
            if tipo not in CRIADORES:
                raise ValueError(f'Tipo de publicacao desconhecido: "{tipo}".')
            return CRIADORES[tipo]()
```

Arquivo: `forum/criacional/builder.py`

```
        """Padrao BUILDER (criacional).

        Intencao
            Separar a construcao de um objeto complexo da sua representacao, de
            modo que o mesmo processo de construcao possa criar representacoes
            diferentes.

        Aplicacao no forum
            Abrir um topico envolve muitas partes opcionais: titulo, autor, texto
            de abertura, lista de tags e sinalizacao de fixado. Um construtor com
            cinco parametros opcionais seria ilegivel; o Builder monta o topico
            passo a passo, valida o que e obrigatorio e so entao chama o Factory
            Method para fabricar o produto final.

        Participantes
            Builder / Builder concreto -> ConstrutorDeTopico
            Produto (Product)          -> Topico
            Diretor (Director)         -> ForumFacade (forum.estrutural.facade)
        """

        from .factory_method import CriadorDeTopico


        class ConstrutorDeTopico:
            """Monta um Topico passo a passo. Todos os metodos sao encadeaveis."""

            def __init__(self):
                self._reiniciar()

            def _reiniciar(self):
                self._identificador = None
                self._titulo = None
                self._autor = None
                self._conteudo = ""
                self._tags = []
                self._fixado = False

            # --- passos de construcao ------------------------------------------------
            def com_identificador(self, identificador):
                self._identificador = identificador
                return self

            def com_titulo(self, titulo):
                titulo = (titulo or "").strip()
                if not titulo:
                    raise ValueError("O titulo do topico nao pode ser vazio.")
                self._titulo = titulo
                return self

            def com_autor(self, autor):
                self._autor = autor
                return self

            def com_conteudo(self, conteudo):
                self._conteudo = conteudo or ""
                return self

            def com_tag(self, tag):
                tag = (tag or "").strip()
                if tag and tag not in self._tags:
                    self._tags.append(tag)
                return self

            def com_tags(self, tags):
                for tag in tags or []:
                    self.com_tag(tag)
                return self
```

```
    def fixado(self, valor=True):
        self._fixado = valor
        return self

    # --- entrega do produto ------------------------------------------------
    def construir(self):
        if self._identificador is None:
            raise ValueError("Builder incompleto: identificador nao informado.")
        if self._autor is None:
            raise ValueError("Builder incompleto: autor nao informado.")
        if self._titulo is None:
            raise ValueError("Builder incompleto: titulo nao informado.")

        print("[Builder] Montando o topico passo a passo:")
        print(f"  - titulo .....: {self._titulo}")
        print(f"  - autor ......: {self._autor.apelido}")
        print(f"  - conteudo ...: {self._conteudo or '(vazio)'}")
        print(f"  - tags .......: {', '.join(self._tags) if self._tags else '(nenhuma)'}")
        print(f"  - fixado .....: {'sim' if self._fixado else 'nao'}")

        topico = CriadorDeTopico().publicar(
            self._identificador,
            self._autor,
            titulo=self._titulo,
            conteudo=self._conteudo,
            tags=self._tags,
        )
        topico.fixado = self._fixado
        self._reiniciar()
        return topico
```

Arquivo: `forum/estrutural/composite.py`

```
"""Padrao COMPOSITE (estrutural).

Intencao
    Compor objetos em estruturas de arvore para representar hierarquias
    todo-parte, permitindo que o cliente trate objetos individuais e
    composicoes de objetos de maneira uniforme.

Aplicacao no forum
    Uma discussao e uma arvore: o Topico contem Mensagens, cada Mensagem
    pode conter respostas (outras Mensagens) e Anexos. Exibir, contar e
    percorrer funcionam da mesma forma em qualquer nivel, porque todos os
    nos implementam a interface ItemDiscussao.

Participantes
    Componente (Component) -> ItemDiscussao
    Folha (Leaf)           -> Anexo
    Composto (Composite)   -> Mensagem, Topico
"""

from abc import ABC, abstractmethod

from ..comportamental.observer import Assunto
from ..comportamental.state import Aberto


class ItemDiscussao(ABC):
    """Componente: interface comum as folhas e aos compostos."""

    def __init__(self, identificador, autor, conteudo):
        self.identificador = identificador
        self.autor = autor
        self.conteudo = conteudo
        self.pai = None

    # --- interface uniforme ------------------------------------------------
    @property
    @abstractmethod
    def tag(self):
        """Prefixo curto usado na composicao do caminho (ex.: T1, M2, A3)."""

    @abstractmethod
    def rotulo(self):
        """Linha de texto que representa o item na arvore."""

    def filhos(self):
        """Folhas devolvem uma colecao vazia; compostos sobrescrevem."""
        return ()

    def adicionar(self, item):
        raise TypeError(f"{type(self).__name__} e uma folha e nao aceita filhos.")

    def remover(self, item):
        raise TypeError(f"{type(self).__name__} e uma folha e nao possui filhos.")

    # --- operacoes recursivas ----------------------------------------------
    def exibir(self, nivel=0):
        print("    " * nivel + self.rotulo())
        for filho in self.filhos():
            filho.exibir(nivel + 1)

    def total_itens(self):
        return 1 + sum(filho.total_itens() for filho in self.filhos())
```

```
    def total_mensagens(self):
        proprio = 1 if isinstance(self, Mensagem) else 0
        return proprio + sum(filho.total_mensagens() for filho in self.filhos())

    def total_votos(self):
        proprio = getattr(self, "votos", 0)
        return proprio + sum(filho.total_votos() for filho in self.filhos())

    def caminho(self):
        """Caminho do no a partir da raiz da discussao (ex.: /T1/M2/A3)."""
        if self.pai is None:
            return f"/{self.tag}"
        return f"{self.pai.caminho()}/{self.tag}"

    def percorrer(self):
        """Percurso em pre-ordem: o proprio no e depois os descendentes."""
        yield self
        for filho in self.filhos():
            yield from filho.percorrer()

    def raiz(self):
        return self if self.pai is None else self.pai.raiz()

    def __str__(self):
        return self.rotulo()


class Anexo(ItemDiscussao):
    """Folha: arquivo anexado a uma mensagem. Nao possui filhos."""

    def __init__(self, identificador, autor, nome_arquivo, tamanho_kb=0):
        super().__init__(identificador, autor, nome_arquivo)
        self.nome_arquivo = nome_arquivo
        self.tamanho_kb = tamanho_kb

    @property
    def tag(self):
        return f"A{self.identificador}"

    def rotulo(self):
        return f"[ANEXO #{self.identificador}] {self.nome_arquivo} ({self.tamanho_kb} KB)"


class NoComposto(ItemDiscussao):
    """Implementacao comum aos nos que armazenam filhos."""

    def __init__(self, identificador, autor, conteudo):
        super().__init__(identificador, autor, conteudo)
        self._filhos = []

    def filhos(self):
        return tuple(self._filhos)

    def adicionar(self, item):
        if not isinstance(item, ItemDiscussao):
            raise TypeError("Somente objetos ItemDiscussao podem ser adicionados.")
        item.pai = self
        self._filhos.append(item)
        return item

    def remover(self, item):
        if item not in self._filhos:
            return False
        self._filhos.remove(item)
        item.pai = None
        return True


class Mensagem(NoComposto):
    """Composto: resposta que pode conter outras respostas e anexos."""

    def __init__(self, identificador, autor, conteudo):
        super().__init__(identificador, autor, conteudo)
        self.votos = 0

    @property
    def tag(self):
        return f"M{self.identificador}"

    def votar(self):
        self.votos += 1
        return self.votos

    def rotulo(self):
        return (
            f"[MSG #{self.identificador}] {self.autor.apelido}: "
            f"{self.conteudo} (votos: {self.votos})"
        )


class Topico(NoComposto, Assunto):
    """Composto raiz da discussao.

    Acumula tres papeis de padroes diferentes:
      - Composite : e a raiz da arvore de mensagens;
      - Observer  : e o Assunto observado pelos usuarios inscritos;
      - State     : delega a um objeto EstadoTopico as regras de resposta.
    """

    def __init__(self, identificador, autor, titulo, conteudo, tags=None):
```

```
        NoComposto.__init__(self, identificador, autor, conteudo)
        Assunto.__init__(self)
        self.titulo = titulo
        self.tags = list(tags or [])
        self.fixado = False
        self._estado = Aberto()

    @property
    def tag(self):
        return f"T{self.identificador}"

    # --- State ------------------------------------------------------------
    @property
    def estado(self):
        return self._estado

    def definir_estado(self, estado):
        self._estado = estado

    def aceita_resposta(self):
        return self._estado.pode_receber_resposta()

    def abrir(self):
        return self._estado.abrir(self)

    def fechar(self):
        return self._estado.fechar(self)

    def arquivar(self):
        return self._estado.arquivar(self)

    # --- Composite --------------------------------------------------------
    def total_respostas(self):
        return self.total_mensagens()

    def rotulo(self):
        marcador = "[FIXADO] " if self.fixado else ""
        etiquetas = f" (tags: {', '.join(self.tags)})" if self.tags else ""
        return (
            f"[TOPICO #{self.identificador}] {marcador}{self.titulo} "
            f"<{self._estado.nome}> - por {self.autor.apelido}{etiquetas}"
        )
```

Arquivo: `forum/estrutural/decorator.py`

```
"""Padrao DECORATOR (estrutural).

Intencao
    Anexar responsabilidades adicionais a um objeto dinamicamente. Os
    decoradores sao uma alternativa flexivel ao uso de heranca para
    estender funcionalidades.

Aplicacao no forum
    O texto publicado passa por filtros que podem ser combinados em
    qualquer ordem e quantidade: censura de palavras proibidas, destaque
    de moderador e assinatura do autor. Criar uma subclasse para cada
    combinacao possivel seria inviavel; os decoradores sao empilhados em
    tempo de execucao.

Participantes
    Componente (Component)   -> Conteudo
    Componente concreto      -> ConteudoSimples
    Decorador (Decorator)    -> DecoradorDeConteudo
    Decoradores concretos    -> ComCensura, ComDestaque, ComAssinatura
"""

import re
from abc import ABC, abstractmethod


class Conteudo(ABC):
    """Componente: tudo que sabe se renderizar como texto."""

    @abstractmethod
    def renderizar(self):
        """Devolve o texto final que sera exibido no forum."""

    def __str__(self):
        return self.renderizar()


class ConteudoSimples(Conteudo):
    """Componente concreto: o texto cru digitado pelo usuario."""

    def __init__(self, texto):
        self._texto = texto

    def renderizar(self):
        return self._texto


class DecoradorDeConteudo(Conteudo):
    """Decorador base: mantem a referencia ao componente envolvido."""

    def __init__(self, conteudo):
        if not isinstance(conteudo, Conteudo):
            raise TypeError("Um decorador so envolve objetos do tipo Conteudo.")
```

```
            self._conteudo = conteudo

        def renderizar(self):
            return self._conteudo.renderizar()


    class ComCensura(DecoradorDeConteudo):
        """Substitui as palavras proibidas por asteriscos."""

        PALAVRAS_PROIBIDAS = ("idiota", "burro", "lixo")

        def renderizar(self):
            texto = super().renderizar()
            for palavra in self.PALAVRAS_PROIBIDAS:
                texto = re.sub(
                    palavra,
                    "*" * len(palavra),
                    texto,
                    flags=re.IGNORECASE,
                )
            return texto


    class ComDestaque(DecoradorDeConteudo):
        """Marca o texto como aviso de moderacao."""

        def renderizar(self):
            return f">>> {super().renderizar()} <<<"


    class ComAssinatura(DecoradorDeConteudo):
        """Acrescenta a assinatura do autor ao final do texto."""

        def __init__(self, conteudo, usuario):
            super().__init__(conteudo)
            self._usuario = usuario

        def renderizar(self):
            return f"{super().renderizar()}\n-- {self._usuario.apelido} ({self._usuario.papel})"


    DECORADORES = {
        "censura": ComCensura,
        "destaque": ComDestaque,
        "assinatura": ComAssinatura,
    }
```

Arquivo: `forum/estrutural/facade.py`

```
    """Padrao FACADE (estrutural).

    Intencao
        Fornecer uma interface unificada para um conjunto de interfaces de um
        subsistema. O Facade define uma interface de nivel mais alto que torna
        o subsistema mais facil de usar.

    Aplicacao no forum
        Abrir uma discussao exige coordenar seis participantes: o repositorio
        unico (Singleton), o construtor passo a passo (Builder), o criador do
        produto (Factory Method), a arvore da discussao (Composite), os
        inscritos (Observer) e o estado inicial (State). O menu do programa
        nao precisa conhecer nada disso: ele chama um unico metodo do Facade.

    Participantes
        Facade     -> ForumFacade
        Subsistema -> Forum, ConstrutorDeTopico, criadores, Topico/Mensagem/Anexo,
                      observadores, estados e estrategias de ordenacao
    """

    from ..comportamental.observer import NotificadorDeUsuario, RegistroDeAtividade
    from ..comportamental.strategy import ESTRATEGIAS, ListagemDeTopicos, PorMaisRecentes
    from ..criacional.builder import ConstrutorDeTopico
    from ..criacional.factory_method import CriadorDeAnexo, CriadorDeMensagem
    from ..criacional.singleton import Forum
    from ..modelos import Usuario
    from .composite import Mensagem, Topico
    from .decorator import DECORADORES, ComAssinatura, ConteudoSimples


    class ForumFacade:
        """Interface unica de alto nivel para todo o subsistema do forum."""

        def __init__(self):
            self._forum = Forum()
            self._construtor = ConstrutorDeTopico()
            self._listagem = ListagemDeTopicos(PorMaisRecentes())
            self._registro = RegistroDeAtividade()

        @property
        def forum(self):
            return self._forum

        @property
        def registro(self):
            return self._registro

        # --- usuarios ------------------------------------------------------
```

```
    def cadastrar_usuario(self, apelido, email, papel="membro"):
        apelido = (apelido or "").strip()
        if not apelido:
            print("Erro: o apelido nao pode ser vazio.")
            return None
        if self._forum.obter_usuario(apelido) is not None:
            print(f'Erro: ja existe um usuario com o apelido "{apelido}".')
            return None
        try:
            usuario = Usuario(apelido, email, papel)
        except ValueError as erro:
            print(f"Erro: {erro}")
            return None
        self._forum.registrar_usuario(usuario)
        print("[Facade] Usuario registrado no repositorio unico (Singleton).")
        print(f'Usuario "{usuario.apelido}" cadastrado como {usuario.papel}.')
        return usuario

    # --- topicos -------------------------------------------------------------
    def abrir_discussao(self, apelido_autor, titulo, conteudo, tags=None, fixado=False):
        autor = self._exigir_usuario(apelido_autor)
        if autor is None:
            return None
        if fixado and not autor.e_moderador:
            print("Erro: somente moderadores podem fixar um topico.")
            return None

        print("[Facade] Coordenando Builder + Factory Method + Singleton + Observer...")
        try:
            topico = (
                self._construtor.com_identificador(self._forum.proximo_identificador())
                .com_titulo(titulo)
                .com_autor(autor)
                .com_conteudo(conteudo)
                .com_tags(tags)
                .fixado(fixado)
                .construir()
            )
        except ValueError as erro:
            print(f"Erro: {erro}")
            return None

        self._forum.adicionar_topico(topico)
        topico.inscrever(NotificadorDeUsuario(autor))
        topico.inscrever(self._registro)
        print("[Facade] Topico armazenado no forum e observadores inscritos.")
        print(f'Topico "{topico.titulo}" aberto com sucesso em {topico.caminho()}.')
        return topico

    def responder(self, identificador_pai, apelido_autor, texto):
        autor = self._exigir_usuario(apelido_autor)
        if autor is None:
            return None
        alvo = self._forum.localizar(identificador_pai)
        if alvo is None:
            print(f"Erro: nenhum item encontrado com o identificador #{identificador_pai}.")
            return None
        if not isinstance(alvo, (Topico, Mensagem)):
            print("Erro: anexos sao folhas da arvore e nao podem receber respostas.")
            return None

        topico = alvo.raiz()
        print(f"[Facade] Verificando o estado do topico #{topico.identificador}...")
        if not topico.aceita_resposta():
            print(f"[State] O topico esta {topico.estado.nome} e nao aceita novas respostas.")
            print("Resposta nao publicada.")
            return None

        mensagem = CriadorDeMensagem().publicar(
            self._forum.proximo_identificador(), autor, conteudo=texto
        )
        alvo.adicionar(mensagem)
        print(f"[Composite] Mensagem inserida em {mensagem.caminho()}.")
        topico.notificar(
            "nova-resposta",
            f'{autor.apelido} respondeu em "{topico.titulo}"',
        )
        print("Resposta publicada com sucesso!")
        return mensagem

    def anexar(self, identificador_mensagem, apelido_autor, nome_arquivo, tamanho_kb=0):
        autor = self._exigir_usuario(apelido_autor)
        if autor is None:
            return None
        alvo = self._forum.localizar(identificador_mensagem)
        if alvo is None:
            print(f"Erro: nenhum item encontrado com o identificador #{identificador_mensagem}.")
            return None
        try:
            anexo = CriadorDeAnexo().publicar(
                self._forum.proximo_identificador(),
                autor,
                nome_arquivo=nome_arquivo,
                tamanho_kb=tamanho_kb,
            )
            alvo.adicionar(anexo)
        except TypeError as erro:
            print(f"Erro: {erro}")
            return None
        print(f"[Composite] Folha inserida em {anexo.caminho()}.")
```

```
                return anexo

            def exibir_discussao(self, identificador_topico):
                topico = self._forum.obter_topico(identificador_topico)
                if topico is None:
                    print(f"Erro: topico #{identificador_topico} nao encontrado.")
                    return False
                print(f"[Composite] Percorrendo a arvore a partir de {topico.caminho()}:")
                topico.exibir()
                print(f"Total de itens na arvore: {topico.total_itens()} "
                      f"(respostas: {topico.total_respostas()}, votos: {topico.total_votos()}).")
                return True

            def listar_topicos(self, chave_estrategia=None):
                if chave_estrategia is not None:
                    if chave_estrategia not in ESTRATEGIAS:
                        print(f'Erro: estrategia desconhecida: "{chave_estrategia}".')
                        return []
                    self._listagem.definir_estrategia(ESTRATEGIAS[chave_estrategia]())

                topicos = self._listagem.listar(self._forum.topicos)
                if not topicos:
                    print("Nenhum topico cadastrado.")
                    return []
                for posicao, topico in enumerate(topicos, start=1):
                    print(f"{posicao}. {topico.rotulo()}")
                    print(f"   respostas: {topico.total_respostas()} | votos: {topico.total_votos()}")
                return topicos

            # --- observer ------------------------------------------------------------
            def inscrever(self, identificador_topico, apelido):
                topico = self._forum.obter_topico(identificador_topico)
                if topico is None:
                    print(f"Erro: topico #{identificador_topico} nao encontrado.")
                    return False
                usuario = self._exigir_usuario(apelido)
                if usuario is None:
                    return False
                if not topico.inscrever(NotificadorDeUsuario(usuario)):
                    print(f'Aviso: "{apelido}" ja estava inscrito neste topico.')
                    return False
                print(f'[Observer] "{apelido}" inscrito. Total de observadores: '
                      f"{len(topico.observadores)}.")
                return True

            def caixa_de_entrada(self, apelido):
                usuario = self._exigir_usuario(apelido)
                if usuario is None:
                    return []
                if not usuario.caixa_entrada:
                    print(f"A caixa de entrada de {apelido} esta vazia.")
                    return []
                print(f"--- Caixa de entrada de {apelido} ---")
                for aviso in usuario.caixa_entrada:
                    print(f"  {aviso}")
                return list(usuario.caixa_entrada)

            # --- state ---------------------------------------------------------------
            ACOES_DE_ESTADO = ("abrir", "fechar", "arquivar")

            def alterar_estado(self, identificador_topico, acao):
                topico = self._forum.obter_topico(identificador_topico)
                if topico is None:
                    print(f"Erro: topico #{identificador_topico} nao encontrado.")
                    return False
                if acao not in self.ACOES_DE_ESTADO:
                    print(f"Erro: acao invalida. Use uma de {self.ACOES_DE_ESTADO}.")
                    return False
                print(f'[Facade] Solicitando a acao "{acao}" ao estado atual '
                      f"({topico.estado.nome})...")
                mudou = getattr(topico, acao)()
                print(f"Estado atual do topico #{topico.identificador}: {topico.estado.nome}.")
                return mudou

            # --- votos ---------------------------------------------------------------
            def votar(self, identificador_mensagem):
                item = self._forum.localizar(identificador_mensagem)
                if item is None:
                    print(f"Erro: nenhum item encontrado com o identificador #{identificador_mensagem}.")
                    return False
                if not isinstance(item, Mensagem):
                    print("Erro: somente mensagens podem receber votos.")
                    return False
                total = item.votar()
                print(f"Voto registrado em {item.caminho()}. Total de votos: {total}.")
                return True

            # --- decorator -----------------------------------------------------------
            def formatar(self, texto, filtros, apelido_autor=None):
                conteudo = ConteudoSimples(texto)
                aplicados = []
                for filtro in filtros:
                    if filtro not in DECORADORES:
                        print(f'Erro: filtro desconhecido: "{filtro}".')
                        return None
                    if filtro == "assinatura":
                        autor = self._exigir_usuario(apelido_autor)
                        if autor is None:
                            return None
                        conteudo = ComAssinatura(conteudo, autor)
```

```
                else:
                    conteudo = DECORADORES[filtro](conteudo)
                aplicados.append(filtro)
            print(f"[Decorator] Pilha aplicada (de dentro para fora): "
                  f"{' -> '.join(aplicados) if aplicados else '(nenhum filtro)'}")
            resultado = conteudo.renderizar()
            print("--- Conteudo renderizado ---")
            print(resultado)
            return resultado

        # --- consultas ---------------------------------------------------------
        def buscar(self, termo):
            termo = (termo or "").strip()
            if not termo:
                print("Erro: o termo de busca nao pode ser vazio.")
                return []
            print(f'[Facade] Buscando "{termo}" em todas as arvores de discussao...')
            encontrados = self._forum.buscar_por_texto(termo)
            if not encontrados:
                print("Nenhum item encontrado.")
                return []
            print(f"{len(encontrados)} item(ns) encontrado(s):")
            for item in encontrados:
                print(f"  {item.caminho()} -> {item.rotulo()}")
            return encontrados

        def estatisticas(self):
            dados = self._forum.estatisticas()
            print("[Singleton] Conferindo a instancia unica: "
                  f"Forum() devolve o mesmo objeto? {Forum() is self._forum}")
            print("--- Estatisticas do forum ---")
            print(f"Usuarios cadastrados ....: {dados['usuarios']}")
            print(f"Topicos abertos .........: {dados['topicos']}")
            print(f"Mensagens publicadas ....: {dados['mensagens']}")
            print(f"Itens na arvore .........: {dados['itens_na_arvore']}")
            print(f"Votos registrados .......: {dados['votos']}")
            print(f"Eventos no registro .....: {len(self._registro.eventos)}")
            print(f"Proximo identificador ...: {dados['proximo_identificador']}")
            return dados

        # --- apoio -------------------------------------------------------------
        def _exigir_usuario(self, apelido):
            usuario = self._forum.obter_usuario(apelido)
            if usuario is None:
                print(f'Erro: usuario "{apelido}" nao cadastrado.')
            return usuario
```

Arquivo: `forum/comportamental/observer.py`

```
"""Padrao OBSERVER (comportamental).

Intencao
    Definir uma dependencia um-para-muitos entre objetos, de modo que
    quando um objeto muda de estado todos os seus dependentes sao
    notificados e atualizados automaticamente.

Aplicacao no forum
    Um topico (Assunto) mantem a lista de usuarios inscritos. Quando uma
    nova resposta e publicada, todos os inscritos (Observadores) recebem
    a notificacao sem que o topico conheca a classe concreta de cada um.

Participantes
    Assunto (Subject)               -> Assunto
    Observador (Observer)           -> Observador
    Observador concreto             -> NotificadorDeUsuario, RegistroDeAtividade
"""

from abc import ABC, abstractmethod


class Observador(ABC):
    """Interface comum a todos os observadores."""

    @abstractmethod
    def atualizar(self, assunto, evento, dados):
        """Reage a um evento emitido pelo assunto observado."""


class Assunto:
    """Mantem a lista de observadores e os notifica."""

    def __init__(self):
        self._observadores = []

    @property
    def observadores(self):
        return tuple(self._observadores)

    def inscrever(self, observador):
        if observador in self._observadores:
            return False
        self._observadores.append(observador)
        return True

    def desinscrever(self, observador):
        if observador not in self._observadores:
            return False
```

```
            self._observadores.remove(observador)
            return True

        def notificar(self, evento, dados):
            total = len(self._observadores)
            print(f'[Observer] Notificando {total} inscrito(s) sobre o evento "{evento}"...')
            if total == 0:
                print("  -> Nenhum inscrito para notificar.")
                return
            for observador in list(self._observadores):
                observador.atualizar(self, evento, dados)


    class NotificadorDeUsuario(Observador):
        """Entrega a notificacao na caixa de entrada de um usuario."""

        def __init__(self, usuario):
            self.usuario = usuario

        def atualizar(self, assunto, evento, dados):
            self.usuario.caixa_entrada.append(f"[{evento}] {dados}")
            print(f"  -> Caixa de entrada de {self.usuario.apelido}: {dados}")

        def __eq__(self, outro):
            return (
                isinstance(outro, NotificadorDeUsuario)
                and outro.usuario.apelido == self.usuario.apelido
            )

        def __hash__(self):
            return hash(("NotificadorDeUsuario", self.usuario.apelido))


    class RegistroDeAtividade(Observador):
        """Observador de auditoria: guarda o historico de eventos do forum."""

        def __init__(self):
            self.eventos = []

        def atualizar(self, assunto, evento, dados):
            self.eventos.append((evento, dados))
            print(f"  -> Registro de atividade: evento {evento} gravado (total: {len(self.eventos)}).")

        def __eq__(self, outro):
            return isinstance(outro, RegistroDeAtividade)

        def __hash__(self):
            return hash("RegistroDeAtividade")
```

Arquivo: `forum/comportamental/strategy.py`

```
    """Padrao STRATEGY (comportamental).

    Intencao
        Definir uma familia de algoritmos, encapsular cada um deles e
        torna-los intercambiaveis. O Strategy permite que o algoritmo varie
        independentemente dos clientes que o utilizam.

    Aplicacao no forum
        A pagina inicial lista os topicos, mas a ordem depende da preferencia
        do usuario: mais recentes, mais respondidos ou mais votados. O
        contexto ListagemDeTopicos nao conhece os algoritmos; ele apenas
        delega a estrategia configurada, que pode ser trocada em execucao.

    Participantes
        Contexto (Context)        -> ListagemDeTopicos
        Estrategia (Strategy)     -> EstrategiaDeOrdenacao
        Estrategias concretas     -> PorMaisRecentes, PorMaisRespondidos, PorMaisVotados
    """

    from abc import ABC, abstractmethod


    class EstrategiaDeOrdenacao(ABC):
        """Interface comum aos algoritmos de ordenacao."""

        nome = "indefinida"

        @abstractmethod
        def ordenar(self, topicos):
            """Devolve uma nova lista de topicos na ordem do algoritmo."""


    class PorMaisRecentes(EstrategiaDeOrdenacao):
        """Identificadores maiores foram criados por ultimo."""

        nome = "mais recentes"

        def ordenar(self, topicos):
            return sorted(topicos, key=lambda t: t.identificador, reverse=True)


    class PorMaisRespondidos(EstrategiaDeOrdenacao):
        nome = "mais respondidos"

        def ordenar(self, topicos):
            return sorted(topicos, key=lambda t: (-t.total_respostas(), t.identificador))
```

```
class PorMaisVotados(EstrategiaDeOrdenacao):
    nome = "mais votados"

    def ordenar(self, topicos):
        return sorted(topicos, key=lambda t: (-t.total_votos(), t.identificador))


ESTRATEGIAS = {
    "recentes": PorMaisRecentes,
    "respondidos": PorMaisRespondidos,
    "votados": PorMaisVotados,
}


class ListagemDeTopicos:
    """Contexto: usa a estrategia configurada sem conhecer o algoritmo."""

    def __init__(self, estrategia=None):
        self._estrategia = estrategia or PorMaisRecentes()

    @property
    def estrategia(self):
        return self._estrategia

    def definir_estrategia(self, estrategia):
        if type(estrategia) is type(self._estrategia):
            print(f"[Strategy] Estrategia mantida: {estrategia.nome}")
            return
        anterior = self._estrategia.nome
        self._estrategia = estrategia
        print(f"[Strategy] Estrategia trocada: {anterior} -> {estrategia.nome}")

    def listar(self, topicos):
        print(f"[Strategy] Ordenando os topicos por: {self._estrategia.nome}")
        return self._estrategia.ordenar(list(topicos))
```

Arquivo: `forum/comportamental/state.py`

```
"""Padrao STATE (comportamental).

Intencao
    Permitir que um objeto altere seu comportamento quando seu estado
    interno muda; o objeto parece mudar de classe.

Aplicacao no forum
    Um topico pode estar Aberto, Fechado ou Arquivado. A permissao para
    receber respostas e as transicoes validas ficam dentro de cada classe
    de estado, eliminando os encadeamentos de "if" espalhados pelo codigo.

Transicoes validas
    Aberto    -> Fechado, Arquivado
    Fechado   -> Aberto, Arquivado
    Arquivado -> (nenhuma: estado terminal)

Participantes
    Contexto (Context)   -> Topico (forum.estrutural.composite)
    Estado (State)       -> EstadoTopico
    Estados concretos    -> Aberto, Fechado, Arquivado
"""

from abc import ABC, abstractmethod


class EstadoTopico(ABC):
    """Interface comum aos estados de um topico."""

    nome = "indefinido"

    @abstractmethod
    def pode_receber_resposta(self):
        """Informa se o topico aceita novas respostas neste estado."""

    def abrir(self, topico):
        return self._recusar(topico, "abrir")

    def fechar(self, topico):
        return self._recusar(topico, "fechar")

    def arquivar(self, topico):
        return self._recusar(topico, "arquivar")

    def _trocar(self, topico, novo_estado):
        print(f"[State] Transicao: {self.nome} -> {novo_estado.nome}")
        topico.definir_estado(novo_estado)
        return True

    def _recusar(self, topico, acao):
        print(f'[State] Transicao recusada: nao e possivel "{acao}" um topico {self.nome}.')
        return False

    def __str__(self):
        return self.nome


class Aberto(EstadoTopico):
```

```
            nome = "Aberto"

            def pode_receber_resposta(self):
                return True

            def abrir(self, topico):
                print("[State] O topico ja esta Aberto.")
                return False

            def fechar(self, topico):
                return self._trocar(topico, Fechado())

            def arquivar(self, topico):
                return self._trocar(topico, Arquivado())


        class Fechado(EstadoTopico):
            nome = "Fechado"

            def pode_receber_resposta(self):
                return False

            def abrir(self, topico):
                return self._trocar(topico, Aberto())

            def fechar(self, topico):
                print("[State] O topico ja esta Fechado.")
                return False

            def arquivar(self, topico):
                return self._trocar(topico, Arquivado())


        class Arquivado(EstadoTopico):
            """Estado terminal: nao permite nenhuma transicao de saida."""

            nome = "Arquivado"

            def pode_receber_resposta(self):
                return False

            def arquivar(self, topico):
                print("[State] O topico ja esta Arquivado.")
                return False
```

---

4. Bateria de testes

Observação sobre a estrutura de dados

O programa inicia com uma única instância de `Forum` (padrão Singleton), criada na primeira chamada a `Forum()` e reaproveitada em todas as demais. Essa instância concentra o dicionário de usuários, o dicionário de tópicos e o contador global de identificadores.

Cada tópico é a raiz de uma árvore (padrão Composite). Tópicos e mensagens guardam a lista de filhos; anexos são folhas e recusam filhos. Todo nó guarda uma referência ao pai, o que permite reconstruir o caminho até a raiz (`/T1/M3/M4/A5`) e descobrir a qual tópico uma mensagem pertence — exatamente o papel dos ponteiros `pai` e `filho` do trabalho em C.

O contador de identificadores é sequencial e **global ao fórum**, e não por tópico: o primeiro tópico recebe `#1`, o segundo `#2`, a primeira mensagem `#3`, e assim por diante. Por isso os identificadores da bateria abaixo não são contínuos dentro de uma mesma árvore.

A sequência completa de entradas desta bateria está no arquivo `testes/roteiro_bateria.txt` e pode ser reproduzida com:

```
python main.py < testes/roteiro_bateria.txt
```

Menu principal

```
===== Forum de Discussao =====
1.  Cadastrar usuario                   [Singleton]
2.  Abrir novo topico                   [Builder + Factory Method]
3.  Responder a um topico ou mensagem   [Factory Method + State + Observer]
4.  Anexar arquivo a uma mensagem       [Composite - folha]
5.  Exibir a arvore de uma discussao    [Composite]
6.  Listar topicos                      [Strategy]
7.  Inscrever usuario em um topico      [Observer]
8.  Alterar o estado de um topico       [State]
9.  Votar em uma mensagem
10. Formatar um conteudo                [Decorator]
11. Buscar por texto                    [Facade]
12. Ver caixa de entrada de um usuario  [Observer]
13. Estatisticas do forum               [Singleton]
0.  Sair
Escolha uma opcao:
```

Teste 1 — Estado inicial: o fórum começa vazio

Objetivo: verificar que o fórum inicia sem nenhum tópico e que a listagem usa a estratégia padrão (mais recentes) mesmo sem dados.

Entrada do usuário:

```
Escolha uma opcao: 6
Ordenar por (recentes/respondidos/votados): recentes
```

Saída esperada do programa:

```
--- Listar Topicos ---
[Strategy] Estrategia mantida: mais recentes
[Strategy] Ordenando os topicos por: mais recentes
Nenhum topico cadastrado.
```

Teste 2 — Cadastro de usuários no repositório único

Objetivo: verificar que os usuários são gravados na instância única de `Forum` e que o papel informado é respeitado.

Entrada do usuário:

```
Escolha uma opcao: 1
Apelido: ana
E-mail: ana@forum.com
Papel (membro/moderador): membro
Escolha uma opcao: 1
Apelido: bruno
E-mail: bruno@forum.com
Papel (membro/moderador): moderador
```

Saída esperada do programa:

```
--- Cadastrar Usuario ---
[Facade] Usuario registrado no repositorio unico (Singleton).
Usuario "ana" cadastrado como membro.

--- Cadastrar Usuario ---
[Facade] Usuario registrado no repositorio unico (Singleton).
Usuario "bruno" cadastrado como moderador.
```

Teste 3 — Apelido duplicado é recusado

Objetivo: verificar que a instância única preserva o cadastro anterior e impede dois usuários com o mesmo apelido.

Entrada do usuário:

```
Escolha uma opcao: 1
Apelido: ana
E-mail: outro@forum.com
Papel (membro/moderador): membro
```

Saída esperada do programa:

```
--- Cadastrar Usuario ---
Erro: ja existe um usuario com o apelido "ana".
```

Teste 4 — Abertura de tópico com Builder e Factory Method

Objetivo: verificar a montagem passo a passo do tópico pelo Builder, a fabricação do produto pelo Factory Method e o registro no fórum com o identificador `#1`.

Entrada do usuário:

```
Escolha uma opcao: 2
Apelido do autor: ana
Titulo: Como funciona a recursao em Python?
Texto de abertura: Nao entendi o caso base.
Tags (separadas por virgula): python,recursao
Fixar o topico? (s/n): n
```

Saída esperada do programa:

```
--- Abrir Novo Topico ---
[Facade] Coordenando Builder + Factory Method + Singleton + Observer...
[Builder] Montando o topico passo a passo:
```

```
  - titulo .....: Como funciona a recursao em Python?
  - autor ......: ana
  - conteudo ...: Nao entendi o caso base.
  - tags .......: python, recursao
  - fixado .....: nao
[Factory Method] CriadorDeTopico vai fabricar um(a) Topico...
[Factory Method] Produto criado: [TOPICO #1] Como funciona a recursao em Python? <Aberto> - por ana (tags: python, recursao)
[Facade] Topico armazenado no forum e observadores inscritos.
Topico "Como funciona a recursao em Python?" aberto com sucesso em /T1.
```

Teste 5 — Regra de permissão e tópico fixado

Objetivo: verificar que a fachada valida a permissão antes de construir qualquer objeto (nenhum identificador é consumido na tentativa recusada) e que um moderador consegue fixar o tópico.

Entrada do usuário:

```
Escolha uma opcao: 2
Apelido do autor: ana
Titulo: Regras do forum
Texto de abertura: Leia antes de postar.
Tags (separadas por virgula): avisos
Fixar o topico? (s/n): s
Escolha uma opcao: 2
Apelido do autor: bruno
Titulo: Regras do forum
Texto de abertura: Leia antes de postar.
Tags (separadas por virgula): avisos
Fixar o topico? (s/n): s
```

Saída esperada do programa:

```
--- Abrir Novo Topico ---
Erro: somente moderadores podem fixar um topico.

--- Abrir Novo Topico ---
[Facade] Coordenando Builder + Factory Method + Singleton + Observer...
[Builder] Montando o topico passo a passo:
  - titulo .....: Regras do forum
  - autor ......: bruno
  - conteudo ...: Leia antes de postar.
  - tags .......: avisos
  - fixado .....: sim
[Factory Method] CriadorDeTopico vai fabricar um(a) Topico...
[Factory Method] Produto criado: [TOPICO #2] Regras do forum <Aberto> - por bruno (tags: avisos)
[Facade] Topico armazenado no forum e observadores inscritos.
Topico "Regras do forum" aberto com sucesso em /T2.
```

Teste 6 — Inscrição de um observador no tópico

Objetivo: verificar que o tópico acumula observadores. O autor e o registro de atividade são inscritos automaticamente na abertura; `bruno` é o terceiro.

Entrada do usuário:

```
Escolha uma opcao: 7
Identificador do topico (#): 1
Apelido do usuario: bruno
```

Saída esperada do programa:

```
--- Inscrever Usuario em Topico ---
[Observer] "bruno" inscrito. Total de observadores: 3.
```

Teste 7 — Resposta publicada e notificação automática

Objetivo: verificar a consulta ao estado antes de publicar, a fabricação da mensagem `#3`, a inserção na árvore e a notificação de todos os inscritos.

Entrada do usuário:

```
Escolha uma opcao: 3
Identificador do topico ou da mensagem (#): 1
Apelido do autor: bruno
Resposta: Use um caso base para parar a recursao.
```

Saída esperada do programa:

```
--- Responder ---
[Facade] Verificando o estado do topico #1...
[Factory Method] CriadorDeMensagem vai fabricar um(a) Mensagem...
```

```
[Factory Method] Produto criado: [MSG #3] bruno: Use um caso base para parar a recursao. (votos: 0)
[Composite] Mensagem inserida em /T1/M3.
[Observer] Notificando 3 inscrito(s) sobre o evento "nova-resposta"...
  -> Caixa de entrada de ana: bruno respondeu em "Como funciona a recursao em Python?"
  -> Registro de atividade: evento nova-resposta gravado (total: 1).
  -> Caixa de entrada de bruno: bruno respondeu em "Como funciona a recursao em Python?"
Resposta publicada com sucesso!
```

Teste 8 — Resposta aninhada e anexo (folha da árvore)

Objetivo: verificar que uma mensagem pode receber respostas (nó composto) e anexos (nó folha), criando profundidade na árvore. A resposta é dirigida à mensagem `#3`, e não ao tópico.

Entrada do usuário:

```
Escolha uma opcao: 3
Identificador do topico ou da mensagem (#): 3
Apelido do autor: ana
Resposta: Obrigada! Funcionou aqui.
Escolha uma opcao: 4
Identificador da mensagem (#): 4
Apelido do autor: ana
Nome do arquivo: saida.txt
Tamanho em KB: 2
```

Saída esperada do programa:

```
--- Responder ---
[Facade] Verificando o estado do topico #1...
[Factory Method] CriadorDeMensagem vai fabricar um(a) Mensagem...
[Factory Method] Produto criado: [MSG #4] ana: Obrigada! Funcionou aqui. (votos: 0)
[Composite] Mensagem inserida em /T1/M3/M4.
[Observer] Notificando 3 inscrito(s) sobre o evento "nova-resposta"...
  -> Caixa de entrada de ana: ana respondeu em "Como funciona a recursao em Python?"
  -> Registro de atividade: evento nova-resposta gravado (total: 2).
  -> Caixa de entrada de bruno: ana respondeu em "Como funciona a recursao em Python?"
Resposta publicada com sucesso!

--- Anexar Arquivo ---
[Factory Method] CriadorDeAnexo vai fabricar um(a) Anexo...
[Factory Method] Produto criado: [ANEXO #5] saida.txt (2 KB)
[Composite] Folha inserida em /T1/M3/M4/A5.
```

Teste 9 — Percurso recursivo da árvore da discussão

Objetivo: verificar que `exibir()` percorre a árvore em pré-ordem, indentando por nível, e que a contagem recursiva considera todos os nós.

Entrada do usuário:

```
Escolha uma opcao: 5
Identificador do topico (#): 1
```

Saída esperada do programa:

```
--- Exibir Arvore da Discussao ---
[Composite] Percorrendo a arvore a partir de /T1:
[TOPICO #1] Como funciona a recursao em Python? <Aberto> - por ana (tags: python, recursao)
   [MSG #3] bruno: Use um caso base para parar a recursao. (votos: 0)
      [MSG #4] ana: Obrigada! Funcionou aqui. (votos: 0)
         [ANEXO #5] saida.txt (2 KB)
Total de itens na arvore: 4 (respostas: 2, votos: 0).
```

Teste 10 — Troca de estratégia de ordenação em tempo de execução

Objetivo: registrar dois votos na mensagem `#3` e verificar que a mesma lista de tópicos muda de ordem apenas por troca da estratégia, sem alterar os dados.

Entrada do usuário:

```
Escolha uma opcao: 9
Identificador da mensagem (#): 3
Escolha uma opcao: 9
Identificador da mensagem (#): 3
Escolha uma opcao: 6
Ordenar por (recentes/respondidos/votados): recentes
Escolha uma opcao: 6
Ordenar por (recentes/respondidos/votados): votados
```

Saída esperada do programa:

```
--- Votar em Mensagem ---
Voto registrado em /T1/M3. Total de votos: 1.

--- Votar em Mensagem ---
Voto registrado em /T1/M3. Total de votos: 2.

--- Listar Topicos ---
[Strategy] Estrategia mantida: mais recentes
[Strategy] Ordenando os topicos por: mais recentes
1. [TOPICO #2] [FIXADO] Regras do forum <Aberto> - por bruno (tags: avisos)
   respostas: 0 | votos: 0
2. [TOPICO #1] Como funciona a recursao em Python? <Aberto> - por ana (tags: python, recursao)
   respostas: 2 | votos: 2

--- Listar Topicos ---
[Strategy] Estrategia trocada: mais recentes -> mais votados
[Strategy] Ordenando os topicos por: mais votados
1. [TOPICO #1] Como funciona a recursao em Python? <Aberto> - por ana (tags: python, recursao)
   respostas: 2 | votos: 2
2. [TOPICO #2] [FIXADO] Regras do forum <Aberto> - por bruno (tags: avisos)
   respostas: 0 | votos: 0
```

Teste 11 — Ciclo de vida do tópico e estado terminal

Objetivo: verificar as transições válidas (Aberto → Fechado → Arquivado), a recusa de resposta em tópico fechado e a impossibilidade de reabrir um tópico arquivado.

Entrada do usuário:

```
Escolha uma opcao: 8
Identificador do topico (#): 1
Acao (abrir/fechar/arquivar): fechar
Escolha uma opcao: 3
Identificador do topico ou da mensagem (#): 1
Apelido do autor: ana
Resposta: Posso perguntar mais uma coisa?
Escolha uma opcao: 8
Identificador do topico (#): 1
Acao (abrir/fechar/arquivar): arquivar
Escolha uma opcao: 8
Identificador do topico (#): 1
Acao (abrir/fechar/arquivar): abrir
```

Saída esperada do programa:

```
--- Alterar Estado do Topico ---
[Facade] Solicitando a acao "fechar" ao estado atual (Aberto)...
[State] Transicao: Aberto -> Fechado
Estado atual do topico #1: Fechado.

--- Responder ---
[Facade] Verificando o estado do topico #1...
[State] O topico esta Fechado e nao aceita novas respostas.
Resposta nao publicada.

--- Alterar Estado do Topico ---
[Facade] Solicitando a acao "arquivar" ao estado atual (Fechado)...
[State] Transicao: Fechado -> Arquivado
Estado atual do topico #1: Arquivado.

--- Alterar Estado do Topico ---
[Facade] Solicitando a acao "abrir" ao estado atual (Arquivado)...
[State] Transicao recusada: nao e possivel "abrir" um topico Arquivado.
Estado atual do topico #1: Arquivado.
```

Teste 12 — Pilha de decoradores aplicada ao conteúdo

Objetivo: verificar que três filtros são empilhados sobre o mesmo texto e aplicados de dentro para fora: primeiro a censura, depois o destaque, por último a assinatura.

Entrada do usuário:

```
Escolha uma opcao: 10
Texto original: Esse post e um lixo, autor burro.
Filtros (censura/destaque/assinatura, por virgula): censura,destaque,assinatura
Apelido para assinatura (enter se nao usar): bruno
```

Saída esperada do programa:

```
--- Formatar Conteudo ---
[Decorator] Pilha aplicada (de dentro para fora): censura -> destaque -> assinatura
--- Conteudo renderizado ---
>>> Esse post e um ****, autor *****. <<<
-- bruno (moderador)
```

Teste 13 — Busca recursiva, caixa de entrada, estatísticas e encerramento

Objetivo: validar a busca que percorre todas as árvores do fórum, confirmar que as notificações do Teste 7 e do Teste 8 chegaram à caixa de entrada de `ana` e conferir que `Forum()` continua devolvendo a mesma instância ao final da execução.

Entrada do usuário:

```
Escolha uma opcao: 11
Termo: recursao
Escolha uma opcao: 12
Apelido do usuario: ana
Escolha uma opcao: 13
Escolha uma opcao: 0
```

Saída esperada do programa:

```
--- Buscar por Texto ---
[Facade] Buscando "recursao" em todas as arvores de discussao...
2 item(ns) encontrado(s):
  /T1 -> [TOPICO #1] Como funciona a recursao em Python? <Arquivado> - por ana (tags: python, recursao)
  /T1/M3 -> [MSG #3] bruno: Use um caso base para parar a recursao. (votos: 2)

--- Caixa de Entrada ---
--- Caixa de entrada de ana ---
  [nova-resposta] bruno respondeu em "Como funciona a recursao em Python?"
  [nova-resposta] ana respondeu em "Como funciona a recursao em Python?"

--- Estatisticas ---
[Singleton] Conferindo a instancia unica: Forum() devolve o mesmo objeto? True
--- Estatisticas do forum ---
Usuarios cadastrados ....: 2
Topicos abertos .........: 2
Mensagens publicadas ....: 2
Itens na arvore .........: 5
Votos registrados .......: 2
Eventos no registro .....: 2
Proximo identificador ...: 6

Encerrando o programa.
A instancia unica do Forum sera descartada pelo coletor de lixo.
```

---

5. Testes automatizados

Além da bateria interativa, o projeto traz 29 testes automatizados em `testes/test_padroes.py`, agrupados por padrão. Cada teste roda com um fórum limpo (`Forum.resetar()`) e com a saída do programa redirecionada, de modo que a verificação é feita sobre o estado dos objetos, e não sobre o texto impresso.

Comando:

```
python -m unittest discover -s testes -v
```

Resultado:

```
test_builder_incompleto (test_padroes.TesteBuilder.test_builder_incompleto) ... ok
test_construcao_encadeada (test_padroes.TesteBuilder.test_construcao_encadeada) ... ok
test_reinicia_apos_construir (test_padroes.TesteBuilder.test_reinicia_apos_construir) ... ok
test_caminho_ate_a_raiz (test_padroes.TesteComposite.test_caminho_ate_a_raiz) ... ok
test_contagem_recursiva (test_padroes.TesteComposite.test_contagem_recursiva) ... ok
test_folha_nao_aceita_filhos (test_padroes.TesteComposite.test_folha_nao_aceita_filhos) ... ok
test_censura (test_padroes.TesteDecorator.test_censura) ... ok
test_ordem_importa (test_padroes.TesteDecorator.test_ordem_importa) ... ok
test_pilha_de_decoradores (test_padroes.TesteDecorator.test_pilha_de_decoradores) ... ok
test_busca_recursiva (test_padroes.TesteFacade.test_busca_recursiva) ... ok
test_fluxo_completo (test_padroes.TesteFacade.test_fluxo_completo) ... ok
test_somente_moderador_fixa (test_padroes.TesteFacade.test_somente_moderador_fixa) ... ok
test_topico_fechado_recusa_resposta (test_padroes.TesteFacade.test_topico_fechado_recusa_resposta) ... ok
test_usuario_duplicado (test_padroes.TesteFacade.test_usuario_duplicado) ... ok
test_cada_criador_devolve_seu_produto (test_padroes.TesteFactoryMethod.test_cada_criador_devolve_seu_produto) ... ok
test_criador_por_nome (test_padroes.TesteFactoryMethod.test_criador_por_nome) ... ok
test_tipo_desconhecido (test_padroes.TesteFactoryMethod.test_tipo_desconhecido) ... ok
test_desinscrever (test_padroes.TesteObserver.test_desinscrever) ... ok
test_nao_duplica_inscricao (test_padroes.TesteObserver.test_nao_duplica_inscricao) ... ok
test_notifica_inscritos (test_padroes.TesteObserver.test_notifica_inscritos) ... ok
test_contador_nao_reinicia (test_padroes.TesteSingleton.test_contador_nao_reinicia) ... ok
test_estado_compartilhado (test_padroes.TesteSingleton.test_estado_compartilhado) ... ok
test_instancia_unica (test_padroes.TesteSingleton.test_instancia_unica) ... ok
test_arquivado_e_terminal (test_padroes.TesteState.test_arquivado_e_terminal) ... ok
test_estado_inicial (test_padroes.TesteState.test_estado_inicial) ... ok
test_fechar_bloqueia_respostas (test_padroes.TesteState.test_fechar_bloqueia_respostas) ... ok
test_mais_recentes (test_padroes.TesteStrategy.test_mais_recentes) ... ok
test_mais_respondidos (test_padroes.TesteStrategy.test_mais_respondidos) ... ok
test_troca_em_execucao (test_padroes.TesteStrategy.test_troca_em_execucao) ... ok

----------------------------------------------------------------------
```

```
Ran 29 tests in 0.004s

OK
```